

# DIGITAL LOGIC DESIGN

## ( CE\_118 )

---

### CHAPTER 4:

# *PROCESSOR DESIGN*

COMPUTER ENGINEERING

## *(part\_1)*

# Outline

## ➤ We introduced new concepts:

- ✓ Instruction sets
- ✓ Instruction types
- ✓ Addressing modes
- ✓ Instruction-execution cycle
- ✓ Processor design flow

## ➤ Including

- ✓ instruction set design,
- ✓ instruction set flowcharts,
- ✓ component allocation,
- ✓ ASM charts
- ✓ processor architecture

## ➤ We have demonstrated processor design

- ✓ 16-bit CISC design
- ✓ 32-bit RISC design
  - data-forwarding
  - branch prediction

# Outline

## ➤ We introduced new concepts:

- ✓ Instruction sets
- ✓ Instruction types
- ✓ Addressing modes
- ✓ Instruction-execution cycle
- ✓ Processor design flow

## ➤ Including

- ✓ instruction set design,
- ✓ instruction set flowcharts,
- ✓ component allocation,
- ✓ ASM charts
- ✓ processor architecture

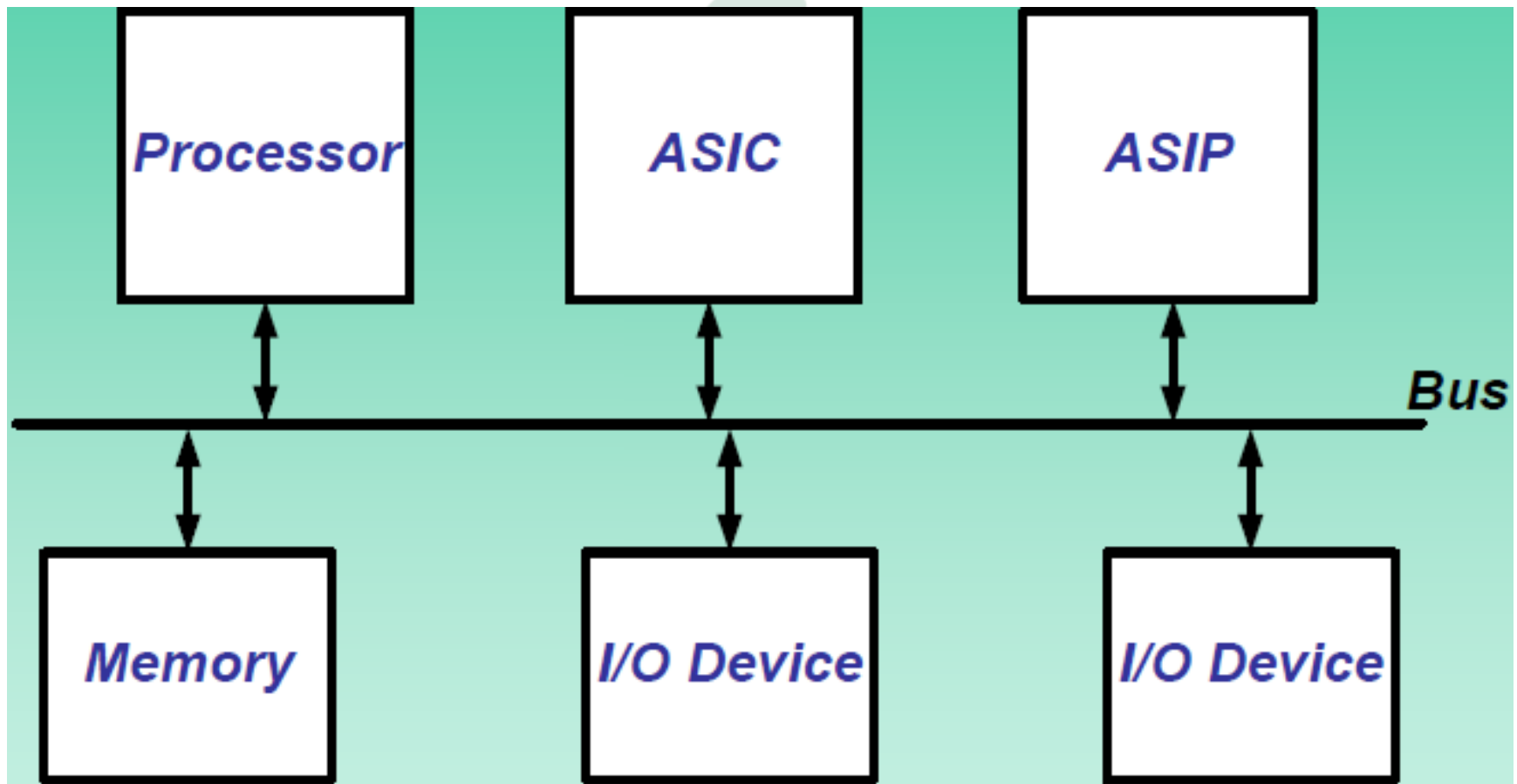
## ➤ We have demonstrated processor design

- ✓ 16-bit CISC design
- ✓ 32-bit RISC design
  - data-forwarding
  - branch prediction

# Basic definitions

- **Processor** controls overall system operations, supervising I/O devices (keyboards, monitors, disks, tapes), synchronize the data between system and components and perform most of computational tasks.
- **ASIC** (Application Specification Integrated Circuit) is a co-processor that executes one or more specific tasks much faster than the processor itself. For this reason, such ASICs are called accelerators, since the processor offloads computationally intensive tasks to them.
- **ASIP** (Application Specification Instruction Processor) is a processor that possesses *specific instructions* that will allow some applications to execute much faster than on an ordinary processor.

# Basic computer architecture





# Typical instructions: instruction-type field

- **Register instructions (arithmetic, logic, shift)**
  - ✓ **Add** RA, RB, RC    ( $RF[A] \leftarrow RF[B] + RF[C]$  )
- **Move instructions (load and store)**
  - ✓ **Load** R2, A    ( $RF[2] \leftarrow MEM[A]$  )
  - ✓ **Store** A, R2    ( $MEM[A] \leftarrow RF[2]$  )
- **Control instructions (branch instruction)**
  - ✓ **Beq** R2, R3, A     $\left\{ \begin{array}{l} \text{if } R2=R3 \text{ then } PC \leftarrow MEM[A] \\ \text{if } R2 \neq R3 \text{ then } PC \leftarrow PC+1 \end{array} \right.$

# Typical instructions: instruction-type field

## ➤ Mode field

**Lind** R2, A      (  $RF[2] \leftarrow Mem[ Mem[A] ]$  )

## ➤ Constant field

**Add** R2, R3, 1    (  $RF[2] \leftarrow RF[3] + 1$  )

COMPUTER ENGINEERING



# Number of address fields vs. performance

- Mathematical expression:  $C = (a+b)(a-b)$
  - Code with **three-address instructions**:
    1. **Add** X, A, B      (  $\text{Mem}[X] \leftarrow \text{Mem}[A] + \text{Mem}[B]$  )
    2. **Sub** C, A, B      (  $\text{Mem}[C] \leftarrow \text{Mem}[A] - \text{Mem}[B]$  )
    3. **Mul** C, X, C      (  $\text{Mem}[C] \leftarrow \text{Mem}[X] \times \text{Mem}[C]$  )
  - Code size: 3 instructions
  - Code performance:
    - Assume memory sizes have 16 to 256 million words → each address would require 24 to 28 bits → 3-address in each instruction would require 80 to 90 bits (including the opcode field)
    - Assume that processor has 32-bit data and memory with 32-bit words → each instruction address would occupy *three words* in memory
- **9** memory accesses to fetch 3 instructions and **9** memory accesses to get the operand and store the result.

# Number of address fields vs. performance

- Mathematical expression:  $C = (a+b)(a-b)$
- Code with **two-address instructions** (with temporary data in memory):
  1. **Move** X, A      (  $\text{Mem}[X] \leftarrow \text{Mem}[A]$  )
  2. **Add** X, B      (  $\text{Mem}[X] \leftarrow \text{Mem}[X] + \text{Mem}[B]$  )
  3. **Move** C, A      (  $\text{Mem}[C] \leftarrow \text{Mem}[A]$  )
  4. **Sub** C, B      (  $\text{Mem}[C] \leftarrow \text{Mem}[C] - \text{Mem}[B]$  )
  5. **Mul** C, X      (  $\text{Mem}[C] \leftarrow \text{Mem}[C] \times \text{Mem}[X]$  )
- Code size: 5 instructions
- Code performance: **10** memory accesses for instructions, **13** memory accesses for data

# Number of address fields vs. performance

- Mathematical expression:  $C = (a+b)(a-b)$
- Code with **one-address instructions** (with temporary data in the **accumulator** (ACC) register):
  1. **Load A**                    (  $ACC \leftarrow Mem[A]$  )
  2. **Add B**                    (  $ACC \leftarrow ACC + Mem[B]$  )
  3. **Store X**                    (  $Mem[X] \leftarrow ACC$  )
  4. **Load A**                    (  $ACC \leftarrow Mem[A]$  )
  5. **Sub B**                     (  $ACC \leftarrow ACC - Mem[B]$  )
  6. **Mul X**                     (  $ACC \leftarrow ACC \times Mem[X]$  )
  7. **Store C**                    (  $Mem[C] \leftarrow ACC$  )
- Code size: 7 instructions
- Code performance: **7** memory accesses for instructions, **7** memory accesses for data

# Number of address fields vs. performance

- Mathematical expression:  $C = (a+b)(a-b)$
- Code with **two-address instructions** (with temporary data in the **register file**):
  1. **Load** R1, A                      (  $RF[1] \leftarrow Mem[A]$  )
  2. **Load** R2, B                      (  $RF[2] \leftarrow Mem[B]$  )
  3. **Move** R3, R2                    (  $RF[3] \leftarrow RF[2]$  )
  4. **Add** R1, R2                    (  $RF[1] \leftarrow RF[1] + RF[2]$  )
  5. **Sub** R3, R2                    (  $RF[3] \leftarrow RF[3] - RF[2]$  )
  6. **Mul** R1, R3                    (  $RF[1] \leftarrow RF[1] \times RF[3]$  )
  7. **Store** C, R1                    (  $Mem[C] \leftarrow RF[1]$  )
- Code size: 7 instructions
- Code performance: **7** memory accesses for instructions,  
**3** (Load x 2, Store) memory accesses for data

## Number of address fields vs. performance

- Mathematical expression:  $C = (a+b)(a-b)$
- Code with **three-address register instructions** and **two-address memory instructions** (with temporary data in the register file):
  1. **Load** R1, A                      ( RF[1]  $\leftarrow$  Mem[A] )
  2. **Load** R2, B                      ( RF[2]  $\leftarrow$  Mem[B] )
  3. **Add** R3, R1, R2                ( RF[3]  $\leftarrow$  RF[1] + RF[2] )
  4. **Sub** R4, R1, R2                ( RF[4]  $\leftarrow$  RF[1] - RF[2] )
  5. **Mul** R5, R4, R3                ( RF[5]  $\leftarrow$  RF[3] x RF[4] )
  6. **Store** C, R5                    ( Mem[C]  $\leftarrow$  RF[5] )
- Code size: 6 instructions
- Code performance: **6** memory accesses for instructions, **3** memory accesses for data

# Addressing modes

## Implied

(set, reset ACC/status registers)

*Op code*

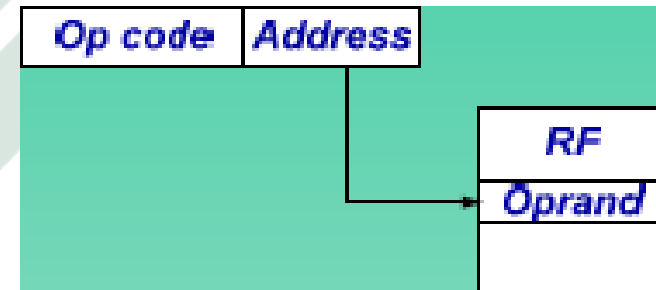
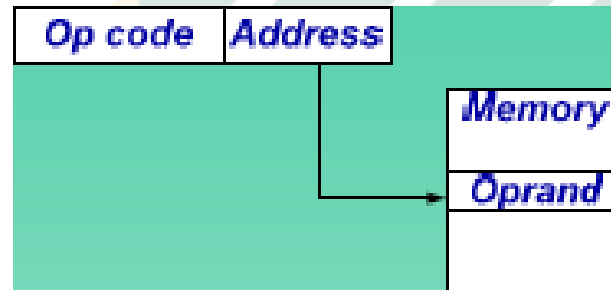
## Immediate

(incre/decre loop,  
array indices, coefficient)

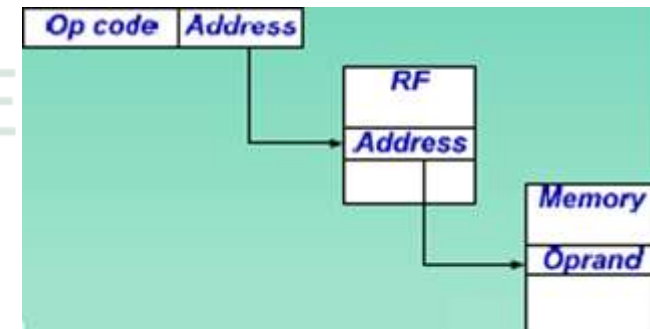
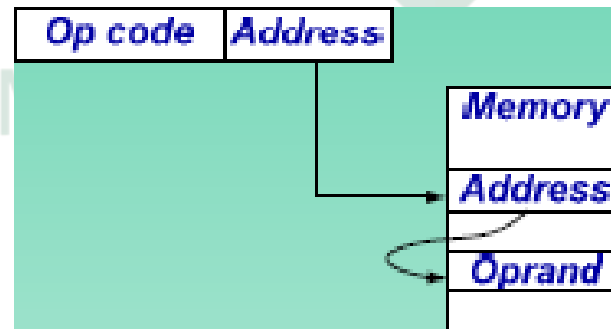
*Op code*

*Oprand*

## Direct

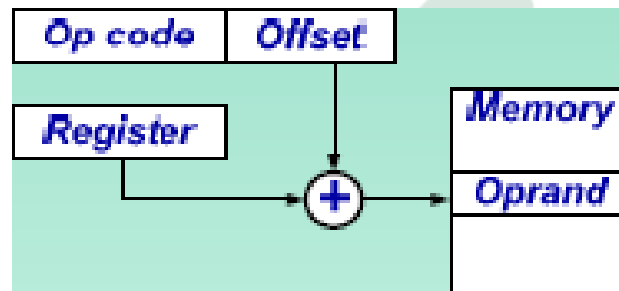


## Indirect



# Addressing modes

## Relative



Branch instructions

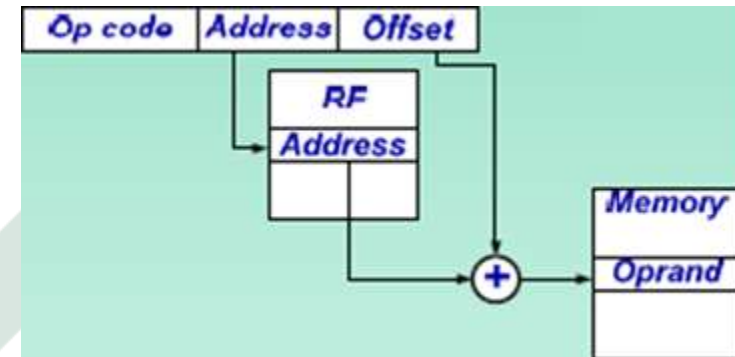
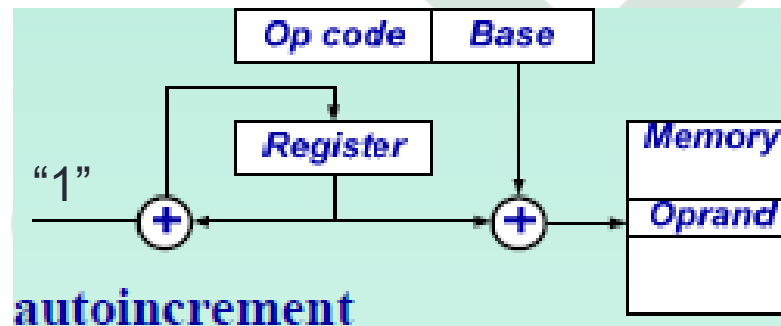
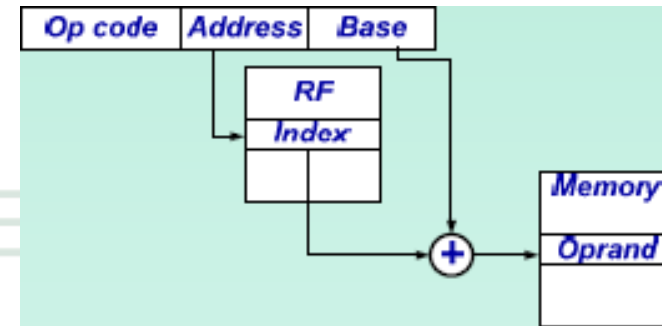


Table look-up

## Indexed



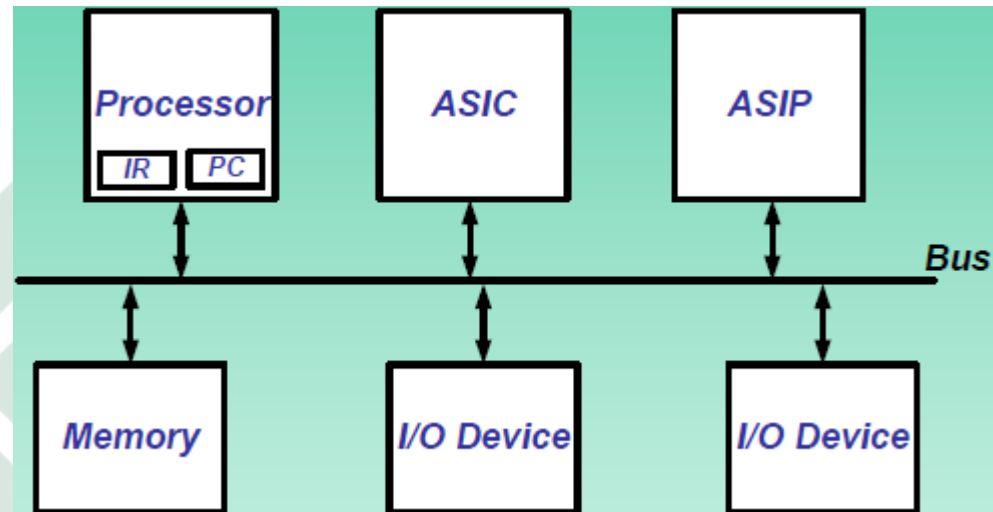
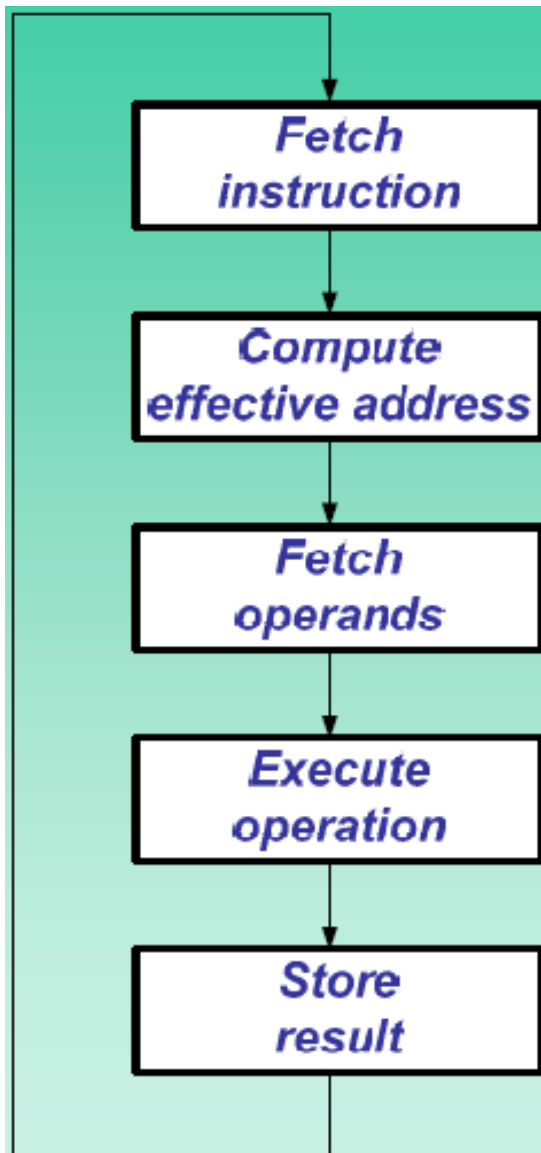
Access stack or queue



Access array

What is the difference between "Relative" and "Indexed" address?

# Instruction-execution cycle



**IR** = Instruction Register

**PC** = Program Counter

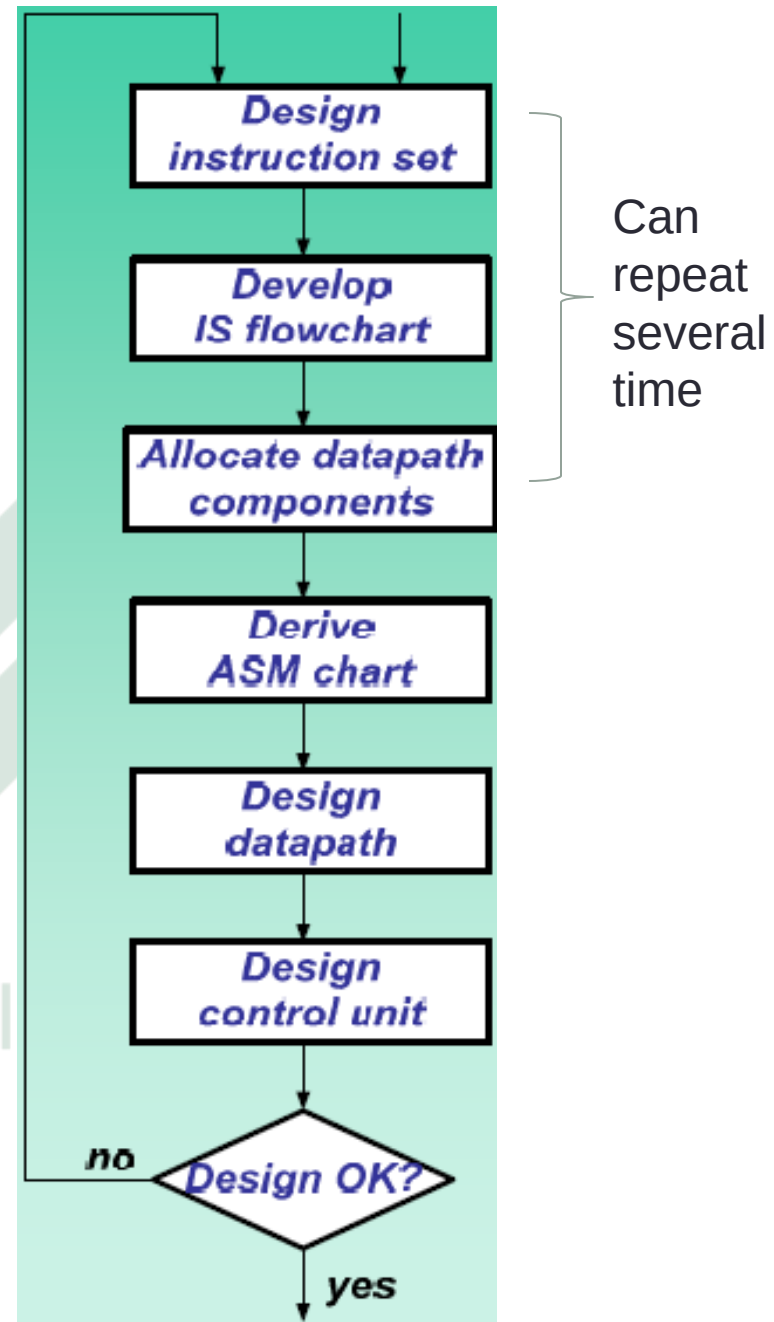
Memory stores all instructions and data



# Processor design flow

Trade-offs between **programming efficiency** and **program size** with **processor cost (size)** and **performance**

- ❖ **Instruction Set (IS)**: specify the operations taken by each instruction.
- ❖ **IS flowchart**: describe precisely all the operations that are performed in each instruction.
- ❖ **ASM chart**: divide each instruction into clock cycle.
- ❖ **Design datapath**: complete the connection.



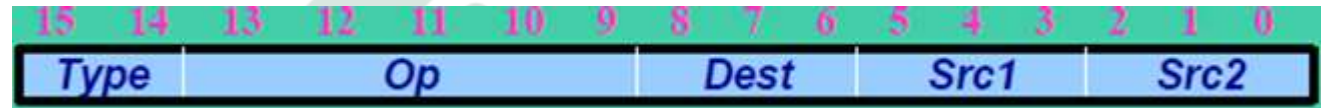
# Instruction-set design

- Programming efficiency vs. program size
- Processor cost vs. processor performance
- Compromise **program size vs. processor size**
- **Complex Instruction-set (CISC)**
  - ✓ powerful instructions -> shorter programs
  - ✓ powerful instructions -> complex datapath, control unit
  - ✓ complex instructions -> several clock cycles
  - ✓ complex datapath, control unit -> longer clock period
  - ✓ complex instructions -> poor pipeline
- **Reduced Instruction-set (RISC)**
  - ✓ simple instructions -> longer programs
  - ✓ simple instructions -> simple datapath, control unit
  - ✓ simple instructions -> single clock cycle
  - ✓ simple datapath -> shorter clock period
  - ✓ simple instructions -> excellent pipeline

# Complex instruction-set for a 16-bit processor

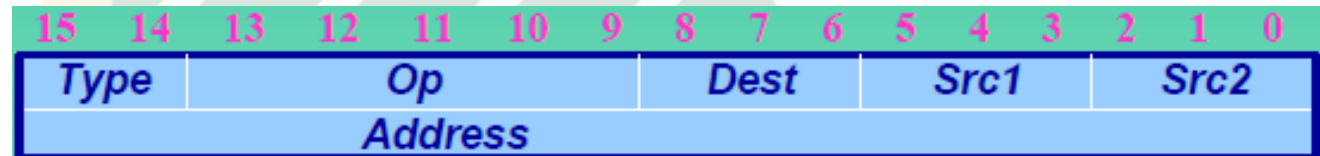
**Each instruction** (different long inst)  $\leftarrow$  **1 word 16-bit** (no memory access) (2-bit Type field, 5-bit Op field,...)  
 $\leftarrow$  **2 words 16-bit** (memory access in word 2)  $\rightarrow$  access **64K** memory

a) **Register instructions**  
 arithmetic, logic,  
 move and shift



| Name                       | Action                                              |
|----------------------------|-----------------------------------------------------|
| <i>Op Dest, Src1, Src2</i> | $RF(Dest) \leftarrow RF(Src1) \text{ Op } RF(Src2)$ |

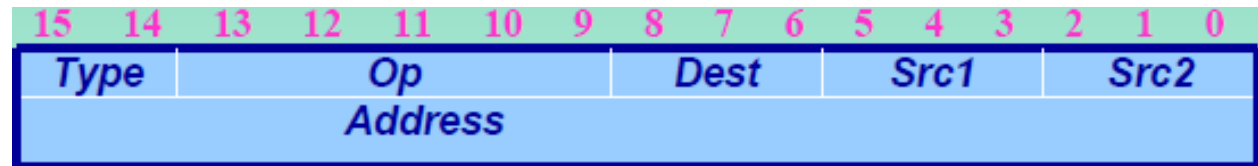
b) **Memory instructions**  
 load and store



| Name                    | Name Action                                 |
|-------------------------|---------------------------------------------|
| <i>L imm Dest</i>       | $RF[Dest] \leftarrow Address$               |
| <i>L dir Dest</i>       | $RF[Dest] \leftarrow Mem[Address]$          |
| <i>L rel Dest, Src2</i> | $RF[Dest] \leftarrow Mem[RF[Src2]+Address]$ |
| <i>L in Dest</i>        | $RF[Dest] \leftarrow Mem[Address]$          |
| <i>S dir Src1</i>       | $Mem[Address] \leftarrow RF[Src1]$          |
| <i>S rel Src1, Src2</i> | $Mem[RF[Src2]+Address] \leftarrow RF[Src1]$ |
| <i>S in Src1</i>        | $Mem[Address] \leftarrow RF[Src1]$          |

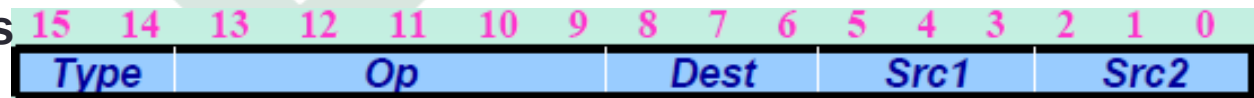
# Complex instruction-set for a 16-bit processor

c) **Control instructions**  
jump, branch, call and return



| Name                      | Action                                                                                                                                         |
|---------------------------|------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Jump</b> Address       | $PC \leftarrow \text{Address}$                                                                                                                 |
| <b>Brel</b> Address       | $PC \leftarrow PC+1$ if $\text{Status}[\text{rel}] = 0$<br>$PC \leftarrow \text{Address}$ if $\text{Status}[\text{rel}] = 1$                   |
| <b>Call</b> Address, Src1 | $\text{Mem}[\text{Src1}] \leftarrow PC+1$ ; $PC \leftarrow \text{Address}$ ;<br>$\text{RF}[\text{Src1}] \leftarrow \text{RF}[\text{Src1}] + 1$ |
| <b>Return</b>             | $\text{RF}[\text{Src1}] \leftarrow \text{RF}[\text{Src1}] - 1$ ;<br>$PC \leftarrow \text{Mem}[\text{Src1}]$ ;                                  |

d) **Miscellaneous instructions**  
no-op, clear, status, set  
and reset



| Name                    | Action                                                                                       | six relations |
|-------------------------|----------------------------------------------------------------------------------------------|---------------|
| <b>No-op</b>            | Do nothing                                                                                   |               |
| <b>Clear</b> Dest       | $\text{RF}[\text{Dest}] \leftarrow 0$                                                        |               |
| <b>Lstat</b> Src1, Src2 | $\text{Status} \leftarrow \text{R}[\text{Src1}] \text{ } \text{>=<} \text{ RF}[\text{Src2}]$ | ↖             |
| <b>Sstat</b> Dest       | $\text{Status}[\text{Dest}] \leftarrow 1$                                                    |               |
| <b>Rstat</b> Dest       | $\text{Status}[\text{Dest}] \leftarrow 0$                                                    |               |

# Bài tập

- Viết các câu lệnh aseembly tính hàm số sau  $y = x^2 + 2x + 3$ , khi biết giá trị  $x$ . Viết các câu lệnh đó với:
  - ✓ Lệnh 3 địa chỉ
  - ✓ Lệnh 2 địa chỉ
  - ✓ Lệnh 1 địa chỉ
- Cho 2 mảng, mỗi mảng 100 phần tử, tính tổng của các phần tử  $\sum a_i x_i$ , Dùng nhóm lệnh 2 địa chỉ (thanh ghi, bộ nhớ), chế độ toán hạng:
  - ✓ Địa chỉ trực tiếp
  - ✓ Địa chỉ gián tiếp

COMPUTER ENGINEERING